N O V A

PERSONAL AI ASSISTANT

Project Architecture & Presentation Guide

A lightweight desktop AI assistant combining voice interaction, Gemini, Whisper speech recognition, text-to-speech, desktop automation, a React/Three.js interface, FastAPI, live system telemetry, and Windows executable packaging.

Portfolio Project Documentation • 2026

1. Project Overview

NOVA was built as a practical desktop AI assistant rather than a simple chatbot. It connects a microphone-driven voice pipeline to an AI reasoning service, converts responses back into speech, executes selected local actions, and presents the system through a custom 3D interface.

Main objectives

- Voice-first interaction instead of a text-only API demo.
- Lightweight runtime suitable for a laptop without keeping a local LLM running continuously.
- Portfolio-quality React and Three.js interface.
- Local backend that coordinates the browser and voice process.
- Windows executable packaging through PyInstaller.

The project demonstrates integration engineering: AI, speech, operating-system interaction, web UI, process management, telemetry, dependency management, and packaging all have to work together.

2. Technology Stack

Layer	Technology	Role
Frontend	React + Vite	Desktop-style web interface
3D	Three.js + React Three Fiber + Drei	Animated NOVA core and visual states
Backend	Python + FastAPI + Uvicorn	Local API and process control
Speech-to-text	OpenAI Whisper	Converts microphone audio to text
Audio	sounddevice	Captures microphone input
AI	Google Gemini	Generates natural-language responses
TTS	pyttsx3	Speaks responses locally
Automation	Python OS utilities / PyAutoGUI	Desktop actions such as screenshots
Telemetry	psutil	RAM, CPU and storage statistics
Packaging	PyInstaller	Builds Windows executable
Launcher	launcher.py	Starts FastAPI and opens the UI

Development note: OpenAI was tested during development but the final working AI path uses Gemini. The OpenAI test hit an insufficient-quota response.

3. System Architecture

NOVA follows a local client-server design. The browser renders the interface, FastAPI provides the local control layer, and the Python voice process performs recording, transcription, AI inference and speech.

USER	Voice input / LISTEN button
↓	
REACT + THREE.JS	3D core • state display • controls • telemetry
↓ HTTP	
FASTAPI / UVICORN	127.0.0.1:8000 • /state • /listen • /system • /health
↓	
VOICE ENGINE	sounddevice → Whisper → command handling → Gemini → pyttsx3
↓	
WINDOWS	Microphone • speakers • desktop actions • system information

The architecture is intentionally modular: the UI does not need to know how Whisper or Gemini works. It only communicates with the local API and renders the resulting state.

4. Voice Processing Pipeline

When NOVA is activated, the voice pipeline follows this sequence:

1. The browser sends a POST request to /listen.
2. FastAPI starts or activates the voice process.
3. sounddevice records microphone audio.
4. Whisper transcribes the recording.
5. NOVA checks whether the text maps to a local command.
6. Conversational requests are sent to Gemini.
7. Gemini returns a text response.
8. pyttsx3 speaks the response.
9. NOVA returns to the ready/idle state.

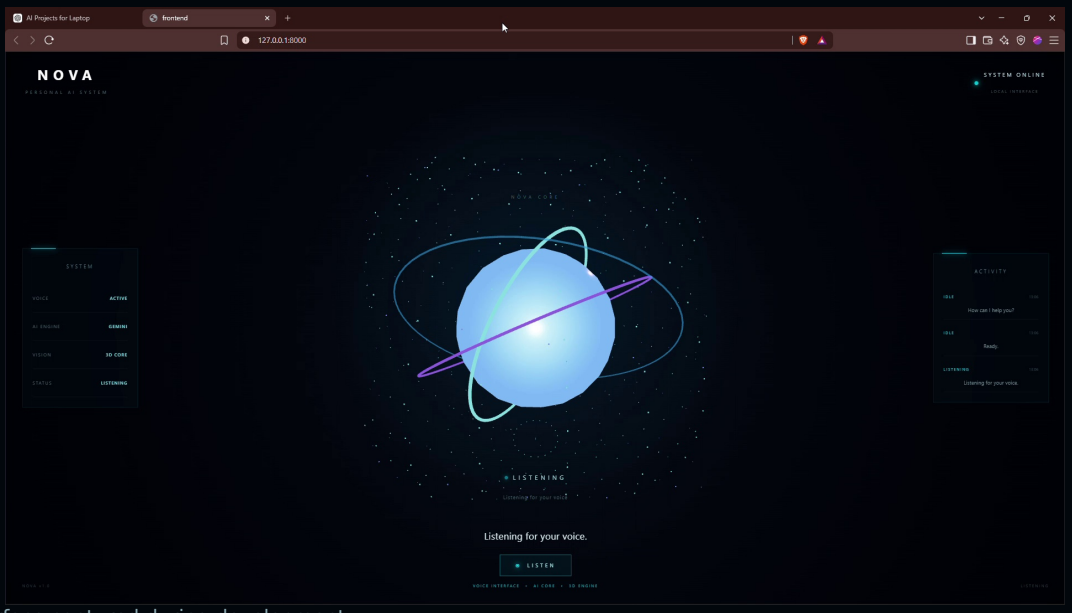
Visual state machine

```
IDLE → LISTENING → THINKING → SPEAKING → IDLE
```

The 3D core reacts to these states, giving immediate feedback while the actual voice/AI work happens in Python.

5. React + Three.js Interface

The frontend is a Vite-powered React application. React manages state and API communication; React Three Fiber renders the Three.js scene; Drei supplies supporting 3D utilities.



NOVA interface captured during development.

- Central animated NOVA core.
- READY / LISTENING / THINKING / SPEAKING indicators.
- LISTEN control connected to the backend.
- System telemetry panel.
- Conversation HUD foundation for future transcript/activity features.

6. Lightweight System Telemetry

Telemetry was designed to feel live without becoming another heavy workload. The backend uses psutil, while React requests data every two seconds. CSS transitions smooth the bars between samples.

Displayed data

- RAM total, used, available and percentage.
- System drive total, used, free and percentage.
- CPU name and current usage.
- GPU name when NVIDIA system information is available.
- Windows version.

```
GET /system
```

The important performance decision is that telemetry is not calculated inside the Three.js render loop. The 3D scene can animate continuously while hardware statistics are sampled only occasionally.

7. Packaging & Startup

The project was developed in VS Code and packaged with PyInstaller. The launcher is the bridge between the packaged application and the local web interface.

Source structure

```
backend/voice_ai.py → voice engine
backend/server.py → FastAPI control layer
frontend/src/* → React interface
launcher.py → startup orchestration
```

Packaged startup

```
NOVA.exe → launcher.py → FastAPI/Uvicorn → browser → React UI → /listen → voice_ai.exe
```

The uploaded launcher imports the FastAPI app and runs Uvicorn on 127.0.0.1:8000, while checking that the production frontend exists before startup.

8. Major Problems Solved

NOVA encountered many of the problems that appear in real software projects. The important fixes included:

- PowerShell execution policy blocking virtual-environment activation.
- Dependencies installed into the wrong Python environment.
- Whisper audio loading requiring FFmpeg.
- Microphone/input-device ambiguity in Windows.
- Native WebRTC VAD build failure because C++ build tools were missing.
- OpenAI quota exhaustion and Gemini temporary 503 availability.
- pyttsx3 and PyAutoGUI/Pillow dependency problems.
- Python control-flow/indentation error with continue.
- PyInstaller missing backend files and Whisper assets.
- Missing Gemini API configuration in the packaged executable.
- Uvicorn port 8000 already being used.
- Windows Smart App Control blocking an unsigned local executable.

Engineering lesson

AI application development is not only model integration. Operating-system permissions, dependency versions, audio hardware, network failures, process lifetime, packaging, security and frontend/backend synchronization all matter.

9. Reliability, Security & Performance

Security

- API keys should be supplied through secure runtime configuration and never committed to source control.
- Desktop automation should use an explicit allow-list instead of executing arbitrary model-generated shell commands.
- An unsigned local executable should not be treated as trusted simply because it was built by the developer.

Performance

- No local LLM is kept running continuously.
- Telemetry is sampled every two seconds.
- The 3D scene uses bounded particle counts and lightweight geometry.
- Voice processing starts only when the user invokes NOVA.

Reliability

External API failures should be caught and converted into friendly fallback responses rather than crashing the assistant.

10. Presentation / Viva Flow

A strong 2–3 minute demonstration can follow this order:

1. Introduce NOVA: “A lightweight desktop personal AI assistant built around voice interaction.”
2. Show the interface: 3D core, status indicators and telemetry.
3. Demonstrate voice: activate LISTEN and ask a question.
4. Explain the pipeline: sounddevice → Whisper → Gemini → pyttsx3.
5. Explain the backend: FastAPI provides the local bridge between UI and Python processes.
6. Explain packaging: PyInstaller packages the application and launcher.py starts the local service.
7. Mention engineering: dependency management, device selection, API failures, packaging paths and Windows security were all handled during development.

One-sentence architecture answer

```
“NOVA is a React and Three.js client connected to a local FastAPI service, which controls a Python voice pipeline using sounddevice, Whisper, Gemini and pyttsx3, with psutil providing lightweight telemetry and PyInstaller packaging the Windows application.”
```

11. Current Status & Future Scope

NOVA is already sufficient as a portfolio project. The current feature set demonstrates AI integration, voice processing, desktop automation, frontend engineering, backend APIs, telemetry and packaging.

Selective future extensions

- Lightweight wake-word activation.
- Activity/history feed.
- More reactive 3D visual feedback tied to voice activity.
- Additional safe desktop commands.
- Signed Windows builds for distribution.

NOVA demonstrates a complete AI application pipeline:

```
INPUT → INTERPRETATION → INTELLIGENCE → ACTION → FEEDBACK
```